

# XIAO ESP32-C5 Pin Map Test

Does the C5 put its Heltec UART where the firmware thinks it does? One test settles it for every fielded C5 Level 2 station at once.

|              |                                                                   |
|--------------|-------------------------------------------------------------------|
| Applies to   | Every fielded Level 2 station on a XIAO ESP32-C5 (remoteid-c5-5g) |
| Not affected | Any ESP32-S3 station, and the carrier PCB itself                  |
| Time         | Test A: 5 min, no hardware · Test B: 15 min, one jumper wire      |
| Date tested  | _____                                                             |

## 1 · What is in question

Two sources disagree about which GPIO sits behind the C5's physical **D4** and **D5** pads — the pads the carrier PCB uses for the Heltec UART.

| SOURCE                                               | D4 (TX → HELTEC RX) | D5 (RX ← HELTEC TX) | ALSO CLAIMS                                 |
|------------------------------------------------------|---------------------|---------------------|---------------------------------------------|
| Current firmware remoteid-c5-5g, hardcoded           | GPIO6               | GPIO7               | —                                           |
| Arduino core variant XIAO_ESP32C5, platform 55.03.39 | GPIO23              | GPIO24              | GPIO6 is the battery-sense pin; GPIO7 is D3 |

The variant looks more likely to be right, on three counts: it ships with the exact platform the project builds against; it puts SDA/SCL on GPIO23/24, matching the XIAO convention that D4/D5 are the I2C pins, as they are on the S3; and it names GPIO6 as the battery-sense pin, which the firmware would be driving as an output. But no one has measured it. That is what this test does.

If the variant is right, every fielded C5 Level 2 station is driving the wrong header pin and its **mesh relay is silent** — the Heltec receives nothing. Such a station still reports normally over USB, which is how this can go unnoticed. Because all Level 2 stations share one carrier layout and one binary, the C5 fleet either all relays or none does.

## 2 · Test A — has any C5 ever relayed? (no hardware)

The cheapest test. Working S3 stations already prove the PCB routes D4/D5 correctly, so the only question is the C5's GPIO mapping — and one historical answer covers the whole fleet.

- ☐ On the collection point, open mapper . Log (or the mapper's console history).
- ☐ Find the node\_id of a C5 Level 2 station. Search the log for it.
- ☐ For each hit, check **which port it arrived on**. A detection relayed over the mesh arrives on the **home node's** serial port. A detection from a C5 plugged directly into the host arrives on that C5's own port.
- ☐ **Question:** has a C5 station's detection *ever* arrived via the home node's port?

| FINDING                                                                     | MEANING                                                                                       |
|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Yes — C5 detections have arrived through the home node                      | GPIO6/GPIO7 is correct. The variant is wrong for this board. <b>Go to sheet 3, outcome 2.</b> |
| No — C5 data only ever arrived on that C5's own USB port                    | Strongly suggests the relay is dead. <b>Confirm with Test B</b> , then sheet 3, outcome 1.    |
| Cannot tell — no C5 has been on the mesh, or logs do not go back far enough | Inconclusive. <b>Go straight to Test B.</b>                                                   |

Test A result: ☐ Yes, relayed ☐ No, never ☐ Cannot tell notes: \_\_\_\_\_

Watch for a trap: a C5 sitting on the host's USB port looks perfectly healthy whichever way this resolves. Only the home node's port proves the mesh path.

### 3 · Test B — measure the pad directly (definitive)

This identifies which GPIO number each physical pad really is. Every candidate pin is configured as an **input**, so nothing is driven and nothing can be damaged.

**You need:** one C5 (a spare on the bench is ideal — no need to pull a station), a USB-C cable, one jumper wire, and PlatformIO or the Arduino IDE.

**Disconnect the battery** before starting, if the board has one attached. GPIO6 is the battery-sense pin on the variant's map, and you will be jumpering pads near it.

#### THE SKETCH

Build for `seeed_xiao_esp32c5` and flash. It only reads pins.

```
/* XIAO pad -> GPIO identifier. SAFE: every candidate pin is an input.
   Jumper ONE pad (e.g. D4) to the 3V3 pad, then read the serial output. */
#include <Arduino.h>

static const int CAND[] = {0,1,2,3,4,5,6,7,8,9,10,11,12,23,24,25,26,27};
static const int N = sizeof(CAND) / sizeof(CAND[0]);

void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000) {}
  delay(300);
  Serial.println("\n=== XIAO pad -> GPIO identifier ===");
}

void loop() {
  // jumper ONE pad to 3V3, watch for a NEW GPIO
  for (int i = 0; i < N; i++) pinMode(CAND[i], INPUT_PULLDOWN);
  delay(20);
  Serial.print("HIGH: ");
  bool any = false;
  for (int i = 0; i < N; i++) {
    if (digitalRead(CAND[i]) == HIGH) { Serial.printf("GPIO%d ", CAND[i]); any = true; }
  }
  if (!any) Serial.print("(none - check the jumper)");
  Serial.println();
  delay(1000);
}
```

#### PROCEDURE

- ☐ **Baseline first.** Run it with **no jumper** and note every GPIO it reports HIGH. Some pins have pull-ups on the module and will read HIGH on their own. Baseline: \_\_\_\_\_
- ☐ Jumper the **D4** pad to the **3V3** pad. The **new** GPIO that appears, one that was not in the baseline, is D4. Record it below.
- ☐ Move the jumper to **D5**, then **D0**, **D6**, **D7** in turn, recording each.
- ☐ If nothing new appears for a pad, re-seat the jumper and confirm you are on the right pad; a dry joint on the header is the usual cause.

#### RECORD

Expected column is the Arduino variant's claim. Write what you actually measured.

| PAD | USED FOR                                     | VARIANT SAYS | FIRMWARE SAYS  | MEASURED GPIO | VARIANT CORRECT?                                      |
|-----|----------------------------------------------|--------------|----------------|---------------|-------------------------------------------------------|
| D4  | Heltec UART TX — <b>the one that matters</b> | GPIO23       | GPIO6          | _____         | <input type="checkbox"/> Y <input type="checkbox"/> N |
| D5  | Heltec UART RX                               | GPIO24       | GPIO7          | _____         | <input type="checkbox"/> Y <input type="checkbox"/> N |
| D0  | Level 1 only: RX5808 RSSI (ADC)              | GPIO1        | GPIO2 (old L1) | _____         | <input type="checkbox"/> Y <input type="checkbox"/> N |
| D6  | Level 1 only: video CSYNC                    | GPIO11       | — never set    | _____         | <input type="checkbox"/> Y <input type="checkbox"/> N |
| D7  | Level 1 only: video VSYNC                    | GPIO12       | — never set    | _____         | <input type="checkbox"/> Y <input type="checkbox"/> N |

**D4 alone decides the Level 2 question;** the other four matter to Level 1. **Cross-check:** jumper D4 to D5 and re-run — both should appear, no others.

## 4 · What each outcome means

### OUTCOME 1 — D4 MEASURES GPIO23 (THE VARIANT IS RIGHT)

Every fielded C5 Level 2 station has been relaying nothing. The fix is one edit, a rebuild and a reflash.

- ☐ In `remoteid-c5-5g/src/main.cpp`, delete the per-board `#if` block and use the board's own pin labels, so one line serves both chips:

```
const int SERIAL1_TX_PIN = D4;    // -> Heltec RX
const int SERIAL1_RX_PIN = D5;    // <- Heltec TX
```

- ☐ Check the other Level 2 projects for the same hardcoding — `node-mode-dualcore` (remote and home) and `remoteid-mesh-dualcore` use GPIO5/GPIO6 with no board conditional at all, so they are S3-only today and would misbehave on a C5.
- ☐ Rebuild both envs; refresh `firmware/xiao-c5-dualband.bin`.
- ☐ **Reflash every fielded C5 Level 2 station.** Confirm each one's detections now reach the mapper through the home node.
- ☐ Inspect the battery-sense path on one board. The old firmware drove GPIO6 push-pull into a divider; harm is unlikely but the reading will have been junk.
- ☐ No Level 1 change needed — it already takes its pins from the variant.

### OUTCOME 2 — D4 MEASURES GPIO6 (THE FIRMWARE IS RIGHT)

The Arduino variant is wrong for this board revision. Level 2 is fine as it stands; **Level 1 is the one that breaks**, because it trusts the variant.

- ☐ In `level1-analog-fpv/src/config.h`, override the pin macros for the C5 with the measured numbers, keeping the D-label names so the intent stays readable.
- ☐ Apply the whole measured column, not just D4/D5 — D0 drives RSSI and D6/D7 drive the video sync, so a wrong variant breaks detection and the video check too.
- ☐ Reverse the C5 caveat in `level1-station/CLAUDE.md`, the Level 1 README wiring table, and the hardware doc.
- ☐ Correct the warning in main's README wiring section.
- ☐ Report the variant bug upstream to Sseed / pioarduino with the measurements.

### OUTCOME 3 — SOMETHING ELSE

If D4 measures a third GPIO, both sources are wrong. Record the full measured column and treat it as authoritative for both tiers.

**Standing requirement, whichever way this goes.** Level 1 stations must use the **same external pins on the S3 and the C5**, interchangeable in one socket, exactly as the Level 2 stations are. Both outcomes above preserve that: the pin *labels* stay fixed and only the GPIO numbers behind them move. Do not solve a mapping problem by moving a signal to a different physical pad on one board.

## 5 · Result

|                        |                |              |  |
|------------------------|----------------|--------------|--|
| Outcome                | 1 2 3 (circle) | D4 measured  |  |
| Tested by              |                | Date         |  |
| Board serial / station |                | Action taken |  |